

Learning With AI

Two Core Focuses:

- Data Manipulation for AI
- AI Accuracy

Concept 1

Temporal Analysis & Signal Processing

The System as a Signal

We treat logs as a **Stochastic Process**: a sequence of random variables evolving over time.

What Is It? The Stochastic Process

- Each metric is a random variable X_t
- Together they form a sequence: $X_1, X_2, X_3 \dots X_t$
- We analyze how values evolve over time

What I Learned: Concept Drift

The real-world data distribution shifts over time — this is called **Concept Drift**.

- A CPU spike at **3 AM** is an anomaly
- The same spike at **3 PM** is expected behavior
- The model must continuously update its baseline as the system changes
- Static thresholds cannot account for this

How We Implemented: Sliding Traces

The model receives a chunk of consecutive values, letting it see **10 seconds of history at once**.

```
traces = numpy.lib.stride_tricks.sliding_window_view(stream, trace_size)
```

1D telemetry stream → 2D matrix of time traces → fed into the MLP.

Concept 2

Probability & Information Theory

Shannon Entropy measures the average uncertainty in a system as a single number.

$$H(X) = - \sum P(x) \log P(x)$$

Low vs High Entropy

Normal system (low entropy):

INFO:	70%
WARNING:	25%
ERROR:	5%

During an incident (high entropy):

INFO:	10%
WARNING:	30%
ERROR:	30%
CRITICAL:	20%
UNKNOWN:	10%

More event types, more evenly spread — the system has become unpredictable.

What I Learned: An Incident Is a Spike in Chaos

We don't look for specific error messages.

We measure when the **distribution of logs** stops being predictable.

- **Minimum entropy** → one outcome is certain
- **Maximum entropy** → all outcomes equally likely

How We Implemented: Entropy Scoring

For each incoming trace we:

1. Count the frequency of each log type
2. Convert counts into probabilities
3. Calculate entropy using the Shannon formula
4. Compare against a **dynamic threshold**

Anomaly alerts

```
from collections import Counter
counts = Counter(trace)
probs = [c / len(trace) for c in counts.values()]
entropy = -sum(p * np.log2(p) for p in probs)

# Threshold is the rolling mean + 2 standard deviations
threshold = np.mean(entropy_history) + 2 * np.std(entropy_history)

if entropy > threshold:
    flag_incident()
```

Concept 3

Probabilistic Classification

The MLP produces one raw score per incident type. Softmax converts all of them into a probability distribution.

What Is It? The Softmax Function

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum e^{x_j}}$$

Each score is divided by the **sum of all scores** — so every output is a probability relative to the others.

Why All Scores at Once?

The MLP outputs one logit per incident type:

CPU Overload:	4.2
Memory Leak:	1.1
Network Failure:	0.3
Disk I/O Error:	2.7

Softmax normalizes all of them together into probabilities that sum to 1.0:

CPU Overload:	0.72
Memory Leak:	0.08
Network Failure:	0.02
Disk I/O Error:	0.15

What I Learned: Using Confidence Scores

- 0.51 → "Uncertain" — escalate for human review
- 0.99 → "Critical Outage" — trigger automated response

How We Implemented: Custom Output Layer

```
exp_scores = np.exp(logits)
softmax = exp_scores / np.sum(exp_scores)
```

One output neuron per incident type, 12 total. Softmax makes all 12 comparable.

Results

By combining **Stochastic Time-Series Analysis** with **Information Theory**:



98.42% Accuracy Rate